

Multithread

Contents

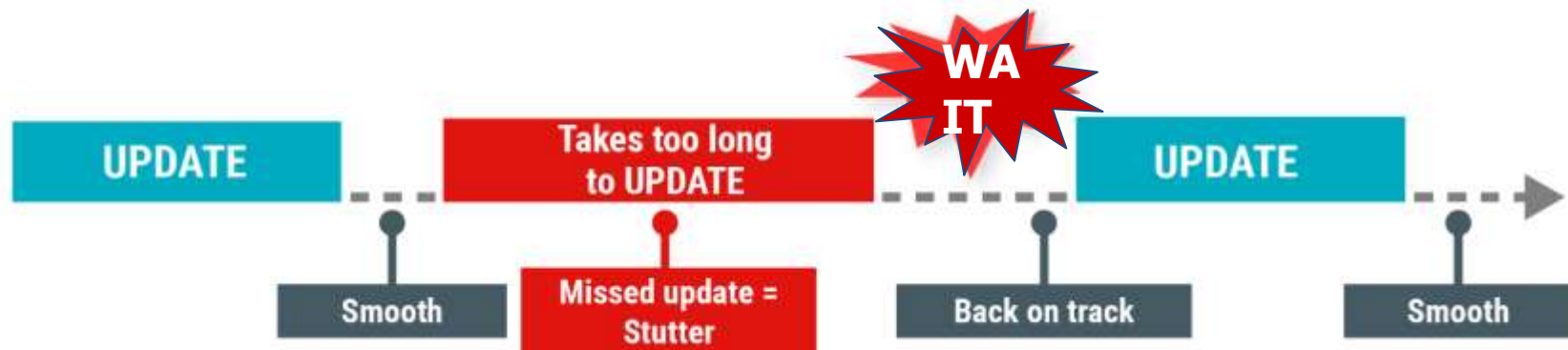
- **Main thread**
- **Worker thread**
- **Some multithread methods**

Main thread

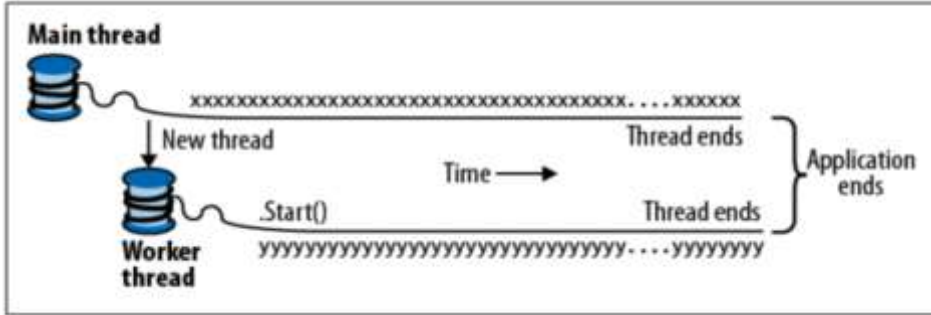
- Independent path of execution in a running program
- Code is executed line by line
- App runs on Java thread called "main" or "UI thread"
- Draws UI on the screen
- Responds to user actions by handling UI events

Main thread

- Hardware updates screen every 16 milliseconds
- UI thread has 16 ms to do all its work
- If it takes too long, app stutters or hangs



Worker thread



Worker threads do:

- Network operations
- Long calculations
- Downloading/uploading files
- Processing images
- Loading data

Two rules for Android threads

- **Do not block the UI thread**
 - Complete all work in less than 16 ms for each screen
 - Run slow non-UI work on a non-UI thread
- **Do not access the Android UI toolkit from outside the UI thread**
 - Do UI work only on the UI thread

Some methods to create worker thread and update UI

- **Using thread and runOnUiThread method:** used to execute a piece of code on the main thread, also known as the UI thread
- **Using Executor:** Provides a flexible and robust way to handle background tasks, especially with **ExecutorService** and **Handler** for updating the UI.

Using thread and runOnUiThread method

```
//TODO: Prepare some works
Thread t = new Thread(new Runnable() {
    @Override
    public void run() {
        //TODO: do some long task
        runOnUiThread(new Runnable() {
            @Override
            public void run() {
                //TODO: update UI here
            }
        });
    }
});
t.start();
```

Using Executor

// 1. Create an Executor

```
ExecutorService executorService = Executors.newSingleThreadExecutor();
```

// 2. Create a Handler to post results back to the UI thread

```
Handler handler = new Handler(Looper.getMainLooper());
```

// 3. Execute the task using Executor

```
executorService.execute(new Runnable() {
```

```
    @Override
```

```
    public void run() {
```

```
        // Background task
```

```
        String result = "Kết quả từ Executor";
```

```
        // Post the result back to the UI thread
```

```
        handler.post(new Runnable() {
```

```
            @Override
```

```
            public void run() { // Update UI
```

```
                textView.setText(result);
```

```
            }
```

```
        });
```

```
    }
```

```
});
```

Classwork

- Using `runOnUiThread` or `Executor` to download 1 image from `Url` into an `ImageView`

Code to load an image from url

- ```
URL url = new URL(str);
HttpsURLConnection connection = (HttpsURLConnection)
url.openConnection();
connection.setDoInput(true);
connection.connect();

InputStream inputstream = connection.getInputStream();
BufferedInputStream buffer = new BufferedInputStream(inputstream);

Bitmap bitmap = BitmapFactory.decodeStream(buffer);
return bitmap;
```

# References

- <https://google-developer-training.github.io/android-developer-fundamentals-course-concepts-v2/>